

L15 · Δυναμικός Προγραμματισμός II

Σακίδιο & Μέγιστη Κοινή Υπακολουθία

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Το πρόβλημα του σακιδίου (0/1 knapsack) με διαδιάστατο DP και ψευδοπολυωνυμική πολυπλοκότητα, και η μέγιστη κοινή υπακολουθία (LCS) δύο συμβολοσειρών.

1 Τι θα δούμε

Δύο κλασικά DP προβλήματα — και ένα **νέο** δίδαγμα. Στο προηγούμενο μάθημα τα υποπροβλήματα παραμετροποιήθηκαν με **μία** μεταβλητή (j = πρώτα j αιτήματα). Εδώ θα δούμε ότι αυτό **δεν αρκεί πάντα**: το πρόβλημα του **σακιδίου** χρειάζεται **δύο** μεταβλητές. Μετά, η **μέγιστη κοινή υπακολουθία** δείχνει DP πάνω σε δύο συμβολοσειρές.

2 Το Πρόβλημα του Σακιδίου (Knapsack)

2.1 Διατύπωση

Είσοδος: n αντικείμενα και ένα «σακίδιο». Το αντικείμενο i έχει **βάρος** $w_i > 0$ και **αξία** $v_i > 0$. Το σακίδιο αντέχει το πολύ W κιλά.

Στόχος: διάλεξε υποσύνολο αντικειμένων που **χωράει** ($\sum w_i \leq W$) και **μεγιστοποιεί** τη συνολική αξία.

Προσοχή

Ο άπληστος αλγόριθμος αποτυγχάνει. Φυσικό κριτήριο: διάλεξε κάθε φορά το αντικείμενο με τον μεγαλύτερο λόγο **αξίας προς βάρος** v_i/w_i . Αλλά αυτός ο λόγος αγνοεί τη χωρητικότητα: ένα αντικείμενο με τέλειο λόγο μπορεί να «σπαταλήσει» χώρο που θα έδινε καλύτερο **συνδυασμό**. (Παράδειγμα: τα αντικείμενα 3, 4 δίνουν αξία 40 με βάρος 11, ενώ τα 5, 2, 1 δίνουν αξία 35.)

2.2 Πρώτη απόπειρα — και γιατί αποτυγχάνει

Δοκιμάζουμε το μοτίβο του προηγούμενου μαθήματος: $\text{OPT}(i)$ = μέγιστη αξία υποσυνόλων από τα αντικείμενα $1, \dots, i$.

- Το i **ΕΞΩ**: $\text{OPT}(i) = \text{OPT}(i - 1)$.
- Το i **ΜΕΣΑ**: η επιλογή του i δεν αποκλείει άμεσα κανένα άλλο αντικείμενο...αλλά **καταναλώνει χώρο**. Χωρίς να ξέρουμε πόσος χώρος έχει ήδη ξοδευτεί από τα προηγούμενα αντικείμενα, **δεν μπορούμε καν να ξέρουμε αν χωράει το i** .

Κλειδί

Η παράμετρος i από μόνη της **δεν περιγράφει πλήρως ένα υποπρόβλημα**. Χρειαζόμαστε να θυμόμαστε και **πόση χωρητικότητα απομένει**. Άρα: μία δεύτερη μεταβλητή.

2.3 Η σωστή αναδρομή — δύο μεταβλητές

Κλειδί

$\text{OPT}(i, w)$ = μέγιστη αξία υποσυνόλου των αντικειμένων $1, \dots, i$ με **συνολικό βάρος το πολύ w** .

Σκεφτόμαστε ξανά το αντικείμενο i :

- Το i **ΕΞΩ**: καλύτερη λύση από τα $1, \dots, i-1$ με χωρητικότητα $w \rightarrow \text{OPT}(i-1, w)$.
- Το i **ΜΕΣΑ** (μόνο αν $w_i \leq w$): «ξοδεύουμε» w_i χώρο και v_i αξία, και λύνουμε το υπόλοιπο με χωρητικότητα $w - w_i \rightarrow v_i + \text{OPT}(i-1, w - w_i)$.

Κλειδί

$$\text{OPT}(i, w) = \begin{cases} 0 & i = 0 \\ \text{OPT}(i-1, w) & w_i > w \\ \max\{ \text{OPT}(i-1, w), v_i + \text{OPT}(i-1, w - w_i) \} & \text{αλλιώς} \end{cases}$$

Κάθε κελί κοιτάει μόνο την προηγούμενη γραμμή: «πάνω» (το i έξω) και «πάνω-αριστερά κατά w_i » (το i μέσα).

2.4 Ο αλγόριθμος

Συμπληρώνουμε έναν πίνακα $n \times W$ — γραμμή προς γραμμή:

```

Σακίδιο(n, w[], v[], W):
  Για w = 0 έως W: M[0, w] <- 0
  Για i = 1 έως n:
    Για w = 0 έως W:
      Εάν w_i > w:
        M[i, w] <- M[i-1, w]
      Αλλιώς:
        M[i, w] <- max{ v_i + M[i-1, w - w_i], M[i-1, w] }
  επίστρεψε M[n, W]

```

Παράδειγμα. Αντικείμενα (w, v) : $1 = (2, 3)$, $2 = (3, 4)$, $3 = (4, 5)$, $4 = (5, 6)$, χωρητικότητα $W = 8$.

$i \backslash w$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (2,3)	0	0	3	3	3	3	3	3	3
2 (3,4)	0	0	3	4	4	7	7	7	7
3 (4,5)	0	0	3	4	5	7	8	9	9
4 (5,6)	0	0	3	4	5	7	8	9	10

Η βέλτιστη αξία είναι $M[4, 8] = 10$ — αντικείμενα 2 και 4 (βάρος $3 + 5 = 8$, αξία $4 + 6 = 10$).

2.5 Πολυπλοκότητα — μια λεπτή παγίδα

Προσοχή

Ο αλγόριθμος τρέχει σε $\Theta(n \cdot W)$. Φαίνεται «πολυωνυμικό» — αλλά **δεν είναι**. Το W είναι ένας **αριθμός** στην είσοδο· για να τον γράψεις χρειάζεσαι μόνο $\log W$ δυφία. Άρα ο χρόνος $n \cdot W$ είναι **εκθετικός** ως προς το μέγεθος της εισόδου. Λέγεται **ψευδοπολυωνυμικός** (pseudo-polynomial).

Κλειδί

Το **πρόβλημα απόφασης** του σακιδίου είναι **NP-πλήρες** — δεν είναι γνωστός (και πιθανότατα δεν υπάρχει) πραγματικά πολυωνυμικός αλγόριθμος. Υπάρχει όμως πολυωνυμικός **προσεγγιστικός** αλγόριθμος που βρίσκει λύση με αξία το πολύ 0.01% μακριά από το βέλτιστο (FPTAS).

3 Μέγιστη Κοινή Υπακολουθία (LCS)

3.1 Διατύπωση

Είσοδος: δύο συμβολοσειρές $x = x_1x_2 \cdots x_m$ και $y = y_1y_2 \cdots y_n$.

Στόχος: το **μέγιστο μήκος** μιας ακολουθίας $z_1z_2 \cdots z_k$ που είναι **ταυτόχρονα υπακολουθία** και των δύο.

Διαίσθηση

Υπακολουθία $\neq$ υποσυμβολοσειρά. Δεν χρειάζεται να είναι **συνεχόμενη** — απλώς να εμφανίζεται με τη σωστή σειρά. Στο ABCBDAB, το BCAB είναι υπακολουθία. Το LCS μετράει πόσο «όμοιες» είναι δύο ακολουθίες — βάση για το `unix diff` και για τη σύγκριση γονιδιωμάτων.

3.2 Η αναδρομή

Ορίζουμε $\text{OPT}(i, j)$ = μέγιστο μήκος κοινής υπακολουθίας των προθεμάτων $x_1 \cdots x_i$ και $y_1 \cdots y_j$. Κοιτάμε τους **τελευταίους** χαρακτήρες x_i, y_j :

- $x_i = y_j$. Τότε αξίζει να τους ζευγαρώσουμε: αυτός ο κοινός χαρακτήρας είναι το τελευταίο στοιχείο της LCS, και το υπόλοιπο είναι LCS των $x_1 \cdots x_{i-1}$ και $y_1 \cdots y_{j-1}$. Άρα $1 + \text{OPT}(i-1, j-1)$.
- $x_i \neq y_j$. Δεν μπορούν να είναι **και οι δύο** το τελευταίο στοιχείο. Άρα τουλάχιστον ένας περισεύει — η LCS είναι είτε των $x_1 \cdots x_{i-1}, y_1 \cdots y_j$ είτε των $x_1 \cdots x_i, y_1 \cdots y_{j-1}$. Παίρνουμε το $\max$.

Κλειδί

$$\text{OPT}(i, j) = \begin{cases} 0 & i = 0 \text{ ή } j = 0 \\ 1 + \text{OPT}(i-1, j-1) & x_i = y_j \\ \max\{\text{OPT}(i-1, j), \text{OPT}(i, j-1)\} & x_i \neq y_j \end{cases}$$

Κάθε κελί κοιτάει: **διαγώνια** (αν $x_i = y_j$), ή **πάνω/αριστερά** (αν $x_i \neq y_j$).

3.3 Ο αλγόριθμος

```
MKY(m, n, x, y):
  Για j = 0 έως n: M[0, j] <- 0
  Για i = 0 έως m: M[i, 0] <- 0
  Για i = 1 έως m:
    Για j = 1 έως n:
      Εάν x_i = y_j:
        M[i, j] <- 1 + M[i-1, j-1]
      Αλλιώς:
        M[i, j] <- max{ M[i, j-1], M[i-1, j] }
  επιστρέψε M[m, n]
```

Παράδειγμα. $x = ABCB, y = BDCAB$:

	$\emptyset$	B	D	C	A	B
$\emptyset$	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3

Η LCS έχει μήκος $M[4, 5] = 3$ — μια κοινή υπακολουθία είναι το BCB.

Πολυπλοκότητα: ο πίνακας έχει $m \cdot n$ κελιά, κάθε ένα $O(1)$. Συνολικά $\Theta(mn)$.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Σακίδιο (0/1 knapsack)** — μέγιστη αξία αντικειμένων που χωράνε σε χωρητικότητα W . Ο άπληστος (λόγος v/w) αποτυγχάνει.
2. Η μονοδιάστατη παραμετροποίηση $OPT(i)$ **δεν αρκεί** — χρειάζεται δεύτερη μεταβλητή: $OPT(i, w) =$ αξία με αντικείμενα $1 \dots i$ και χωρητικότητα w .
3. Αναδρομή: $\max\{OPT(i-1, w), v_i + OPT(i-1, w - w_i)\}$. Χρόνος $\Theta(nW)$ — **ψευδοπολυωνυμικός**: το πρόβλημα απόφασης είναι **NP-πλήρες**.
4. **Μέγιστη κοινή υπακολουθία (LCS)** — μέγιστο μήκος κοινής (όχι κατ' ανάγκη συνεχόμενης) υπακολουθίας δύο συμβολοσειρών.
5. Αναδρομή: αν $x_i = y_j \rightarrow$ διαγώνια $1 + OPT(i-1, j-1)$ · αλλιώς $\rightarrow$ max από «πάνω» και «αριστερά». Χρόνος $\Theta(mn)$.